

Calm Calendar – Current Project State (June 2026)

Project Goal

Building a Calm Calendar-style application that syncs busy blocks between Google Calendars.

Current implementation:

- Google OAuth via NextAuth
 - Source calendar → Target calendar sync
 - Incremental sync using Google Sync Tokens
 - Real-time updates using Google Watch API webhooks
 - Automatic watch-channel renewal
 - Event creation, update, and deletion propagation
-

Original Sync Architecture

Previously:

- Every sync fetched all source events.
- Compared source and target calendars.
- Re-created busy blocks repeatedly.
- Polling-based logic.
- Expensive and inefficient.

Problems:

- Large API usage.
 - Slow syncs.
 - Duplicate processing.
 - Not truly real-time.
-

New Architecture

1. Sync Tokens

Workflow stores:

- syncToken

Google returns:

- nextSyncToken

Usage:

- First sync = full sync.
- Subsequent syncs = incremental sync.

Result:

- Only changed events are returned.
 - Faster.
 - Fewer API calls.
-

2. Google Watch API

Added watch channels:

- Google pushes notifications.
- No polling required.

Workflow stores:

- watchChannelId
- watchResourceId
- watchExpiration

Webhook endpoint:

```
/api/google/webhook
```

Receives:

- x-goog-resource-id
- x-goog-channel-id
- x-goog-resource-state

```
---
```

```
### 3. Webhook Logic
```

```
Flow:
```

```
1. Google sends notification.
```

2. Match workflow using resourceId.
3. Verify channelId.
4. Ignore stale channels.
5. Run googleSync().
6. Update sync token.

4. Stale Channel Protection

Problem:

Multiple old watch channels existed:

- 7e2e...
- ab99...
- 016f...

All received notifications.

Solution:

Webhook checks:

```
```ts
```

```
if (channelId !== workflow.watchChannelId)
```

Old channels are ignored.

Result:

Only newest channel processes events.

---

## 5. Race Condition Protection

Added:

```
syncInProgress Boolean @default(false)
```

Purpose:

Prevent concurrent webhook syncs.

Logic:

1. Check syncInProgress.
2. If true → skip.
3. If false → set true.
4. Run sync.
5. finally → set false.

Result:

No duplicate event creation caused by concurrent requests.

---

## 6. Watch Renewal

Added:

```
renewExpiringWatchChannels()
```

Purpose:

Google channels expire after ~7 days.

Flow:

1. Find channels expiring within 24 hours.
  2. Create replacement channel.
  3. Update workflow.
- 

## 7. Channel Stop Logic

Added:

```
stopWatchChannel()
```

Uses:

```
POST https://www.googleapis.com/calendar/v3/channels/stop
```

Purpose:

Stop old watch channels before creating replacements.

Current status:

Working.

Observed:

Status 204 = success.

---

## 8. Current Database Fields

Workflow currently contains:

- sourceCal
- targetCal
- syncToken
- watchChannelId
- watchResourceId
- watchExpiration
- syncInProgress

These support:

Single source calendar. Single target calendar.

---

## Current Problem Being Solved

Requirement:

One workflow should support:

Multiple source calendars. Multiple destination calendars.

Example:

Workflow A

Sources: - Personal - Work - Family

Targets: - Availability Calendar - Team Calendar

Current schema only supports:

sourceCal (single) targetCal (single)

Need:

Many-to-one relationship.

---

## Planned New Schema

WorkflowSourceCalendar

Stores:

- workflowId
- calendarId
- syncToken
- watchChannelId
- watchResourceId
- watchExpiration

WorkflowTargetCalendar

Stores:

- workflowId
- calendarId

Workflow becomes:

One workflow → many source calendars

One workflow → many target calendars

---

## Files Added During Watch API Work

googleSync.ts

Responsibilities:

- Incremental sync
- Sync token handling
- Event create/update/delete

googleWatch.ts

Responsibilities:

- Create watch channels
- Stop old channels
- Store watch metadata

/api/google/watch/route.ts

Responsibilities:

- Register watch channels

/api/google/webhook/route.ts

Responsibilities:

- Receive Google notifications
- Validate channel
- Trigger sync

renew-watch route

Responsibilities:

- Renew expiring channels

cron.ts

Originally handled polling sync.

Now largely obsolete because Watch API handles updates.

---

## Next Step

Investigate why Prisma Studio does not show:

- WorkflowSourceCalendar
- WorkflowTargetCalendar

despite Prisma reporting:

"Database schema is up to date."

After resolving schema state:

1. Create source/target calendar tables.

2. Migrate existing workflow data.
3. Refactor sync logic to loop over multiple sources and targets.
4. Refactor watch-channel creation per source calendar.
5. Deploy.